KASBench Full Disclosure Report

Preamble

This Full Disclosure Report (FDR) documents the results obtained from a single execution of the Kubernetes Autoscaling Benchmark (KASBench) conducted from October 14, 2026 at 12:16:29 UTC to October 16, 2026 at 02:17:54 UTC. In this evaluation, the following autoscaler configurations were tested:

- Kubernetes Horizontal Pod Autoscaler (HPA)
- Kubernetes Vertical Pod Autoscaler (VPA)
- Kubernetes Event-Driven Autoscaler (KEDA)
- No autoscaler (control condition)

The purpose of this report is to completely document the benchmark environment, execution procedures, system configuration, and observed results of a full KASBench execution to enable independent verification and reproduction.

KASBench is a comparative benchmark intended to evaluate Kubernetes autoscaling systems under controlled workloads and documented conditions. It measures autoscaler performance using five metrics (θ^S, θ^A, τ^S, τ^A, and $\bar{C}$) and three composite scores (D^S, D^A, and ζ). These metrics and composite scores measure deviation from the ideal, and are in the range 0 (best) to 100 (worst) inclusive. Lower scores represent autoscaler performance closer to the ideal.

Synchronous underprovisioning penalty (θ^S) quantifies the mean deviation above a pre-determined service level objective (SLO) for synchronous requests initiated by a synthetic load generator over the benchmark period. Results are calculated as a percentage in the range 0 to 100, inclusive, using an upper bound of five-times the SLO. Sub-metrics for response-time (θ^S_{RT}) and failure-rate (θ^S_{FR}) are blended to calculateθ^S.

Synchronous underprovisioned time slices (τ^S) measures the percentage of 30-second time slices during which requests exceed the SLO by more than 20%. It is calculated by blending the sub-metrics for response-time (τ^S_{RT}) and failure-rate (τ^S_{FR}).

Asynchronous underprovisioning penalty (θ^A) quantifies the how long asynchronous tasks wait in queue beyond a predetermined baseline (1 second). Asynchronous tasks are triggered indirectly from synchronous requests. Results are calculated as a percentage in the range 0 to 100, inclusive, using an upper bound of five-times the SLO.

Asynchronous underprovisioned time slices (τ^A) measures the percentage of 30-second time slices during which asynchronous tasks exceed the baseline by more than 20%.

Blended mean resource utilization ($\bar{C}$) is the blended percentage of available CPU and memory requested by each autoscaler in excess of a baseline. The baseline is calculated as the minimum resources that could be allocated by any autoscaler in the test.

The metrics θ^S, τ^S, θ^A, τ^A, and $\bar{C}$ represent deviation from the ideal on a scale of 0 to 100, inclusive.

Derived synchronous composite score (D^S) is the weighted Minkowski deviation (distance) of θ^S and τ^S from zero.

Derived asynchronous composite score (D^A) is the weighted Minkowski deviation (distance) of θ^A and τ^A from zero.

Composite score (ζ) is the weighted Minkowski distance of D^S, D^A, and $\bar{C}$.

The evaluation utilizes a completely random design. Each benchmark evaluation comprises multiple trials. The number of trials per autoscaler is indicated in the *Metadata* section of this document. Trials are independent. Each trial is conducted in an identical, individually built, one-time-use virtual private cloud in a randomly selected Amazon Web Services (AWS) availability zone. Autoscalers under test are randomly assigned to a trial and AWS availability zone.

Autoscalers Under Test

This section describes the configuration of each autoscaler under test. Autoscalers are expected to be configured optimally for the benchmark evaluation, including resource requests and limits, number of replicas, and autoscaler-specific configuration.

Kubernetes Horizontal Pod Autoscaler (HPA)

HPA is a threshold-based horizontal autoscaler. Each deployment is configured with a single replica. Initial CPU and memory requests are established by a calibration process using a fixed workload set at the 15th percentile of benchmark workload. CPU and memory limits were set at 6 cores and 6 GiB, an arbitrarily large number to permit bursting. HPA was configured to allow scaling from 1 to 20 pods. CPU and memory thresholds are set at 70%. The scale down stabilization was set at 75 seconds.

Kubernetes Event-Driven Autoscaler (KEDA)

KEDA is a threshold-based horizontal pod autoscaler driven by events or other metrics. In this evaluation, a hybrid configuration was employed, using KEDA for asynchronous microservices and HPA for others. KEDA monitored the depth of the Apache Kafka queues used for asynchronous processing. The lag threshold was set to 10 items in the queue. KEDA was configured with a minimum replica count of 1 and a maximum of 20 replicas. The polling interval was 15 seconds. The cool down period was set at 75 seconds. For asynchronous services, HPA was configured as described for the HPA-only evaluation.

Kubernetes Vertical Pod Autoscaler (VPA)

VPA is a threshold-based vertical pod autoscaler. VPA initial resource requests and limits were the same as HPA. The minimum CPU was set the same as the requested CPU, as lower amounts were found to lead to instability. The minimum memory was set based on experimentation to avoid fatal out-of-memory errors. Each deployment was configured with two replicas to facilitate vertical scaling with no downtime. The Pod Disruption Budget was set at a minimum of 1 replica.

No Autoscaler (control condition)

As the control condition, evaluations are conducted with no autoscaling configured. Deployments are configured with the same requests and limits as the other tests and a fixed replica count of 1.

The Full Disclosure Report (FDR)

The following sections are produced systematically from data captured during the execution of the benchmark evaluation. All material parameters, settings, versions, hashes, and descriptions are included to foster reproducibility.

Benchmark Metadata

Benchmark Name	KASBench
Version	v1.0
Testing Dates	2026-10-14 12:16:29 UTC to 2026-10-16 02:17:54 UTC
start_time	2026-10-14 12:16:29
end_time	2026-10-16 02:17:54
Reporting Date	2026-10-21 14:33:32 UTC
Region	us-east-1
Benchmark Runner	c6a.large (amd64)
Control Plane	m8i.large (amd64)
Worker Nodes AMD	1 x c6a.4xlarge
Worker Nodes ARM	1 x c6g.4xlarge
AMD Worker AMI ID	ami-0244585558b2aebd7
ARM Worker AMI ID	ami-052bbac83b5bb0ab9
AMD Runner AMI ID	ami-03a891c9de365d954
Kubernetes Version	1.36.1
Autoscalers Evaluated	Kubernetes Horizontal Pod Autoscaler (HPA), Kubernetes Event-Driven Autoscaler (KEDA), Kubernetes Vertical Pod Autoscaler (VPA), No autoscaler (control condition)
Trials Per Autoscaler	7

Software artifacts

Benchmark Components

Component	Image_original	ImageID
KASBench Controller	kasbench/kasbench-controller:0.0.6	sha256:0816294976161de3279756584c5ee4061eafb860d80e436ca0a33dc99cedabc4
KASBench Runner	kasbench/kasbench-runner:0.2.26	sha256:8a130fc14b736fc3d6f2add2d305eef1e3a1703ade87cbb3f69efdf3796c1f6d
KASBench Load Generator	kasbench/kasbench-load-generator:0.0.21	sha256:5e6ad9d312140a7f6e19f5792aa68b4a1c12d1e2c6999e98f0bd5f24549bca4d

Helm Charts

Name	Version	URL
aws-ebs-csi-driver	2.67.0	https://kubernetes-sigs.github.io/aws-ebs-csi-driver
aws-efs-csi-driver	4.5.1	https://kubernetes-sigs.github.io/aws-efs-csi-driver/
globeco	0.1.60	https://kasbench.github.io/globeco-helm
prometheus	29.37.0	https://prometheus-community.github.io/helm-charts
keda	2.21.0	https://kedacore.github.io/charts

Stateful Services

Stateful Set	Image
postgres	postgres:18.0 docker.io/library/postgres sha256:41fc5342eefba6cc2ccda736aaf034bbbb7c3df0fdb81516eba1ba33f360162c
apache/kafka	apache/kafka:4.0.0 docker.io/apache/kafka sha256:3f7b939115cd4872e9cee9369d80bd69712fde55f9902f46d793f64848dedc75
mongo	mongo:6.0 docker.io/library/mongo sha256:8b6d8f5bbedb25cb73517b65cf99f13aeb75ad5b157a56c479287a840bbad3ac
redis	redis:7-alpine docker.io/library/redis sha256:858f009f9709ce576febc734aa78b8f6d624b82571f9ddb6bda4377c833b3499

Stateless Services

Deployment	Image
globeco-allocation-service	kasbench/globeco-allocation-service-server:1.0.3 docker.io/kasbench/globeco-allocation-service-server sha256:0e6750db5d9e6227554a346f12d0556ea4c62a6047786f2395a539c74218cd5e
globeco-confirmation-service	kasbench/globeco-confirmation-service:1.0.11 docker.io/kasbench/globeco-confirmation-service sha256:6275443ef40937002caba5156796c6a469b8665ac2bbae7d78b3009e646ffad7
globeco-debug-tools	kasbench/globeco-debug-tools:latest docker.io/kasbench/globeco-debug-tools sha256:9f279ca1ccaf4d118c3de3587bb7946198ac6a14569e3990ea978fdc4f644032
globeco-execution-service	kasbench/globeco-execution-service:1.0.9 docker.io/kasbench/globeco-execution-service sha256:949ea19a52e2a118431f3241b9a10febdb154a406a499faa246194179abfd6a1
globeco-fix-engine	kasbench/globeco-fix-engine:1.0.17 docker.io/kasbench/globeco-fix-engine sha256:ae5cda2fdb69d92b94059cbead3677703c5f2a61da37659855b22d8d895c0450
globeco-order-generation-service	kasbench/globeco-order-generation-service:1.0.6 docker.io/kasbench/globeco-order-generation-service sha256:a7541d05c1e720573394cb41b443067a05bbb7db75e69e240df1808c5fed5f14
globeco-order-service	kasbench/globeco-order-service:1.0.4 docker.io/kasbench/globeco-order-service sha256:1600b3c42ace32ddf0296468173bc835344e4c885e0b528f4d4e71e51f457b7f
globeco-portfolio-accounting-service	kasbench/globeco-portfolio-accounting-service-server:1.0.2 docker.io/kasbench/globeco-portfolio-accounting-service-server sha256:2b5646e0ccd9a5c4bf4bbce2749f6325c668802175cfc551c36c24fa36dee69d
globeco-portfolio-management-portal	kasbench/globeco-portfolio-management-portal:1.0.5 docker.io/kasbench/globeco-portfolio-management-portal sha256:a65a6022cb244c4baf0eac2ad1f8dddf626ec4ecf0481947e639a0924feb1193
globeco-portfolio-service	kasbench/globeco-portfolio-service:1.0.4 docker.io/kasbench/globeco-portfolio-service sha256:b0561599453606420eb4e904f31214c9c62c5a580c5ca95b25e59f67eca16dd7
	kasbench/globeco-portfolio-service-high-cpu:1.0.4 docker.io/kasbench/globeco-portfolio-service-high-cpu sha256:1a7821e35baf1814f2a7fddbd446ca59b2deaa86fe0a822a5400a27fb59be49c
globeco-pricing-service	kasbench/globeco-pricing-service:1.0.2 docker.io/kasbench/globeco-pricing-service sha256:2509c2a2e69712fa9a559b21a6f820ee537300592e6b77794c7135f282c06dc9
globeco-security-service	kasbench/globeco-security-service:1.0.1 docker.io/kasbench/globeco-security-service sha256:7066be0ea6f76bd60d1ba3c24a30e748c9c767a5eb13a2859204ab765e632241
globeco-trade-service	kasbench/globeco-trade-service:1.0.6 docker.io/kasbench/globeco-trade-service sha256:030719935159a3722ae29242134d4c4cff451ad2e82fcb85864d49aa604c9700

Raw Performance Metrics

Summarized Performance Metrics by Trial

Trial ID	Autoscaler	Requests	Mean Response Time (ms)	Failures	Failure Rate	Round-trip Completion (%)
trial0001	VPA	538728	82	14	0.003%	2.9%
trial0002	HPA	544340	76	11	0.002%	1.2%
trial0003	VPA	549082	83	1006	0.183%	2.5%
trial0004	NONE	437793	287	576	0.132%	1.7%
trial0005	HPA	546222	76	12	0.002%	1.2%
trial0006	KEDA	539007	80	112	0.021%	6.6%
trial0007	VPA	538885	74	16	0.003%	2.3%
trial0008	VPA	538097	79	7	0.001%	3.0%
trial0009	HPA	542938	87	14	0.003%	1.3%
trial0010	NONE	396881	237	223	0.056%	2.8%
trial0011	KEDA	538791	84	133	0.025%	6.2%
trial0012	KEDA	539247	92	105	0.019%	7.0%
trial0013	HPA	541826	78	35	0.006%	1.3%
trial0014	KEDA	540732	82	94	0.017%	7.4%
trial0015	KEDA	531534	101	107	0.020%	6.3%
trial0016	NONE	499510	279	447	0.089%	1.3%
trial0017	NONE	400056	251	612	0.153%	2.4%
trial0018	KEDA	543205	92	228	0.042%	6.3%
trial0019	HPA	548058	83	75	0.014%	1.4%
trial0020	VPA	547022	81	12	0.002%	3.0%
trial0021	KEDA	532637	97	138	0.026%	6.8%
trial0022	NONE	528991	166	59	0.011%	1.2%
trial0023	NONE	524332	177	91	0.017%	1.6%
trial0024	HPA	552611	73	19	0.003%	1.1%
trial0025	VPA	548588	79	15	0.003%	2.6%
trial0026	HPA	544396	75	51	0.009%	1.0%
trial0027	NONE	468007	271	548	0.117%	1.6%
trial0028	VPA	545748	80	9	0.002%	2.2%

Summarized Performance Metrics by Autoscaler

Autoscaler	Mean Response Time (ms)	Failure Rate	Round-trip Completion (%)
HPA	78	0.006%	1.2%
KEDA	90	0.024%	6.6%
NONE	238	0.082%	1.8%
VPA	80	0.028%	2.6%

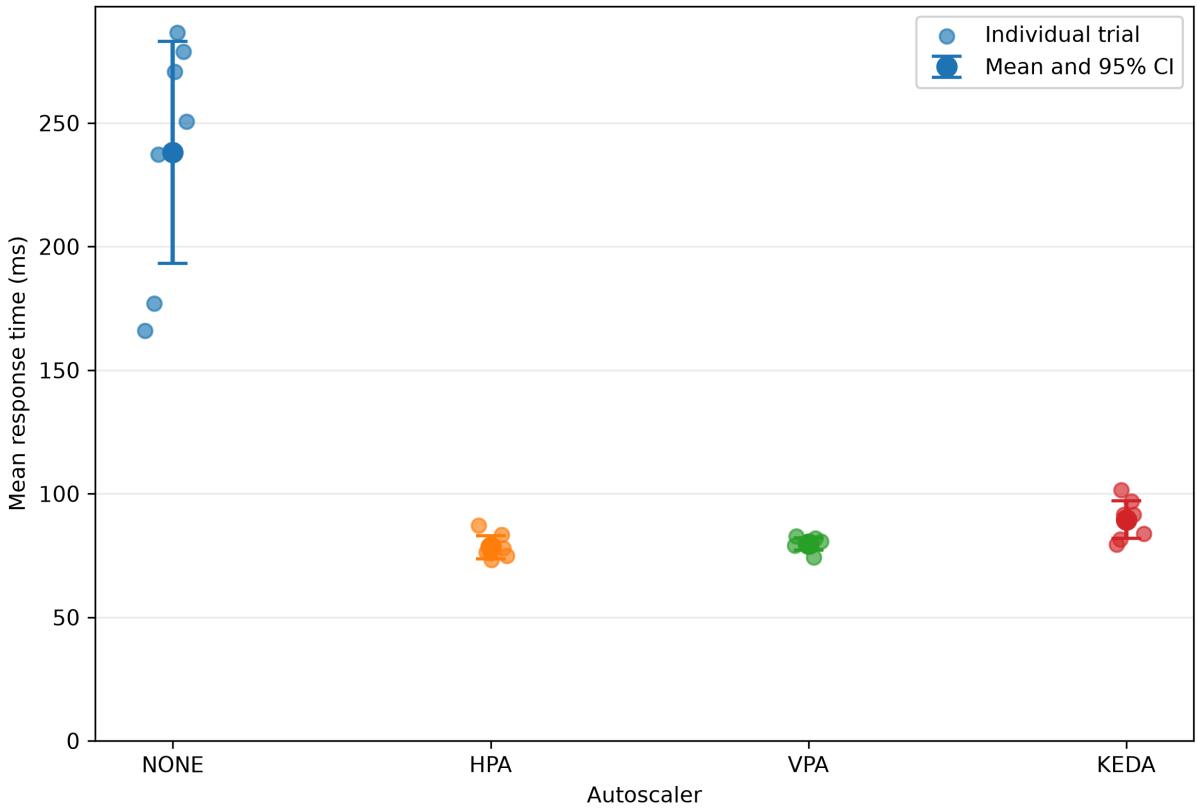


Figure 1. Mean response time by autoscaler.

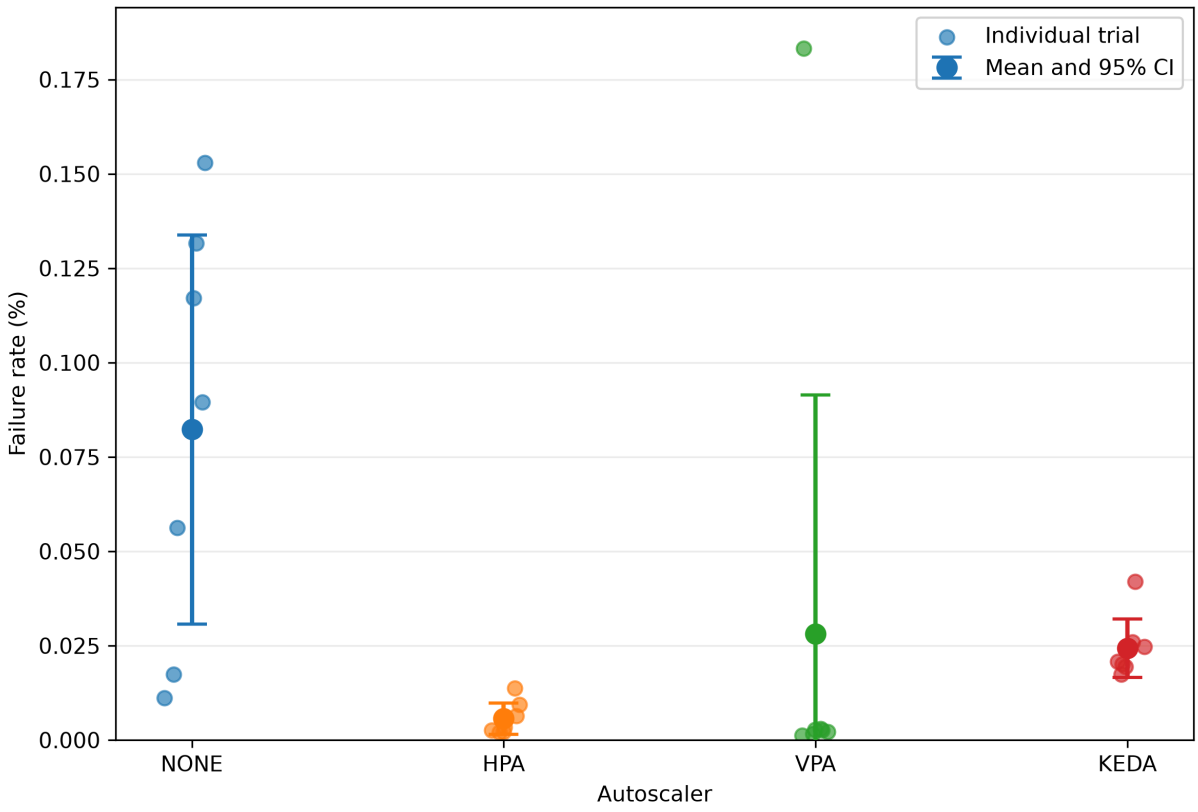


Figure 2. Mean failure rate by autoscaler.

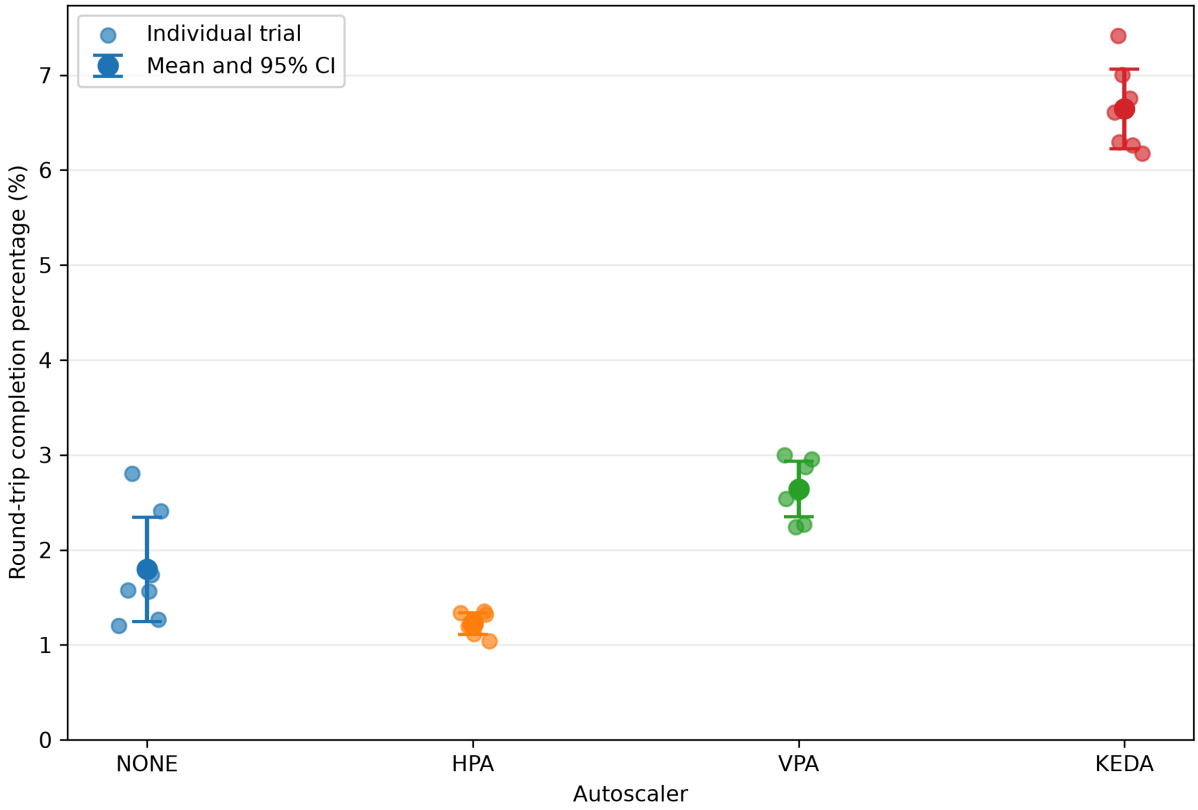


Figure 3. Mean round trip percentage by autoscaler.

Comparative Autoscaler Metrics

Synchronous Underprovisioning Penalty (θ^S) by Autoscaler ($\alpha = 0.5$)

Autoscaler	θ^{RT}	θ^{FR}	θ^S
HPA	40.646	1.576	21.111
KEDA	42.913	3.204	23.059
NONE	71.924	15.165	43.544
VPA	45.554	1.434	23.494

Synchronous Underprovisioned Timeshare (τ^S) by Autoscaler (α = 0.5)

Autoscaler	τ^{RT}	τ^{FR}	τ^S
HPA	89.011	2.599	45.805
KEDA	88.430	4.132	46.281
NONE	91.371	19.846	55.608
VPA	89.138	2.715	45.927

Asynchronous Metrics (θ^A and τ^A) by Autoscaler

Autoscaler	Avg. Lag	θ^A	τ^A
HPA	13345.902	49.989	50.000
KEDA	50.379	12.821	15.486
NONE	7481.291	47.575	48.068
VPA	266.364	31.697	35.013

Cost Metrics by Autoscaler (β = 0.5)

Autoscaler	$\overline{C}_{CPU}$	$\overline{C}_{MEM}$	$\overline{C}$
HPA	41.620	37.314	39.467
KEDA	55.485	45.451	50.468
NONE	13.653	14.614	14.134
VPA	42.135	36.088	39.112

Summary Results by Autoscaler

Autoscaler	θ^S	τ^S	θ^A	τ^A	D^S	D^A	$\overline{C}$	ζ
HPA	21.111	45.805	49.989	50.000	37.505	49.995	39.467	43.688
KEDA	23.059	46.281	12.821	15.486	38.189	14.278	50.468	36.631
NONE	43.544	55.608	47.575	48.068	50.300	47.823	14.134	45.664
VPA	23.494	45.927	31.697	35.013	38.011	33.437	39.112	36.565

Parameters

Parameter	Value
$w_{\theta A}$	0.5
$w_{\tau A}$	0.5
$w_{\theta S}$	0.5
$w_{\tau S}$	0.5
w_S	0.4
w_A	0.4
w_C	0.2
p	3

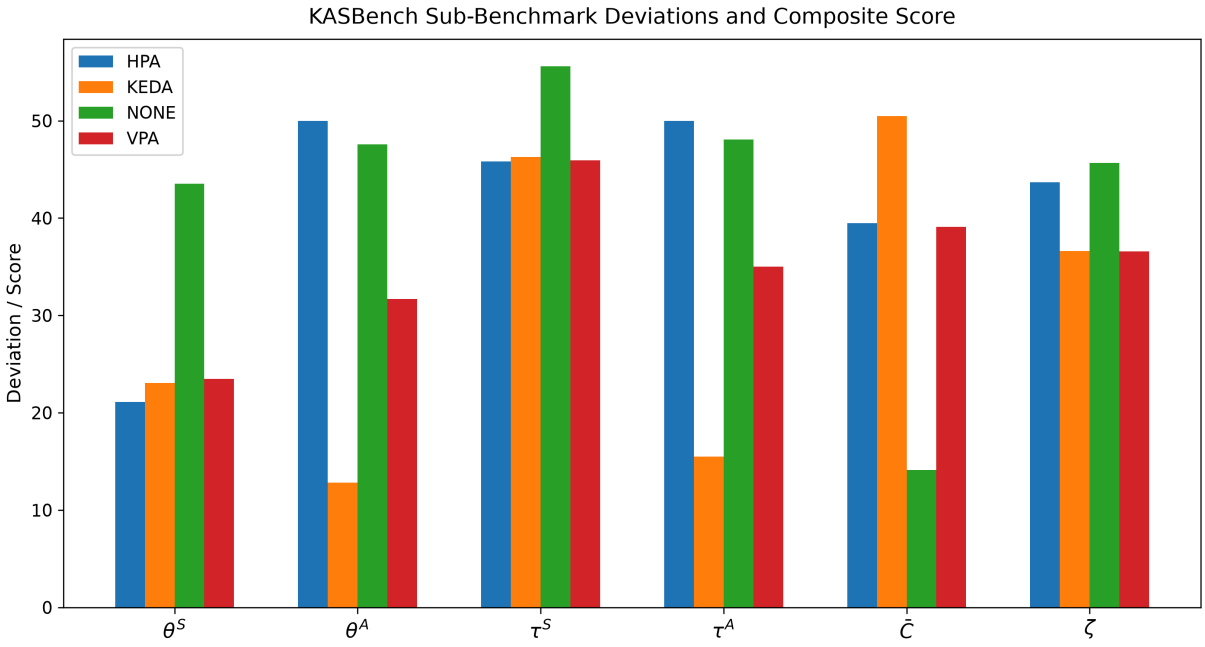


Figure 4. Sub-benchmark deviations and composite score by autoscaler.

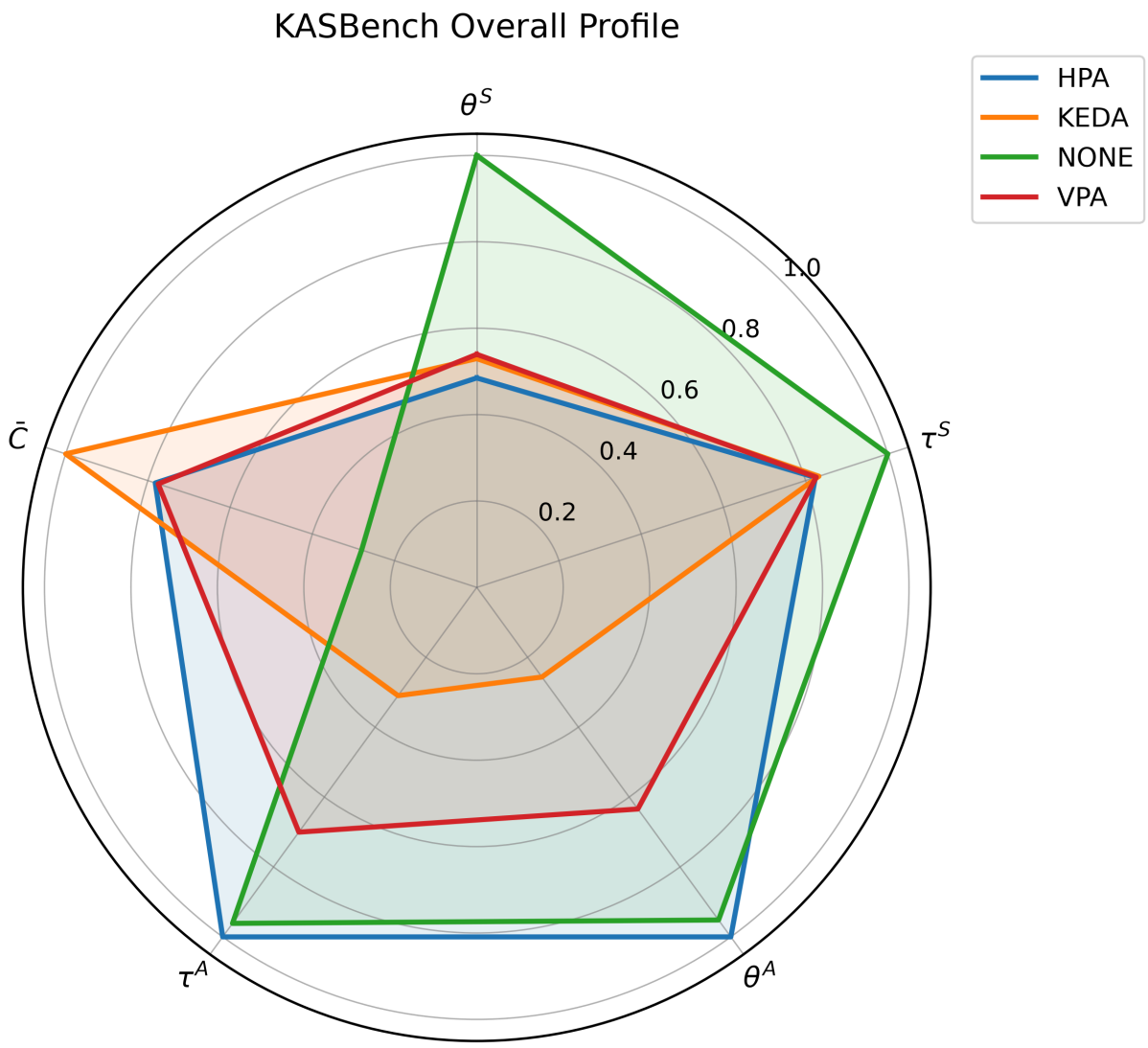


Figure 5. Sub-benchmark deviations by autoscaler.